

Práctica 9: GALARISC-Microarquitectura y Conjunto de Instrucciones

Para esta práctica se representaron circuitos digitales usando Logisim, particularmente para la representación de los componentes de un CPU, ya sea con el program counter, la memoria con selector, la unidad de control y el banco de registros, cuyos circuitos simulan el comportamiento además de las instrucciones que se programaron en C.

1. Program Counter (PC)

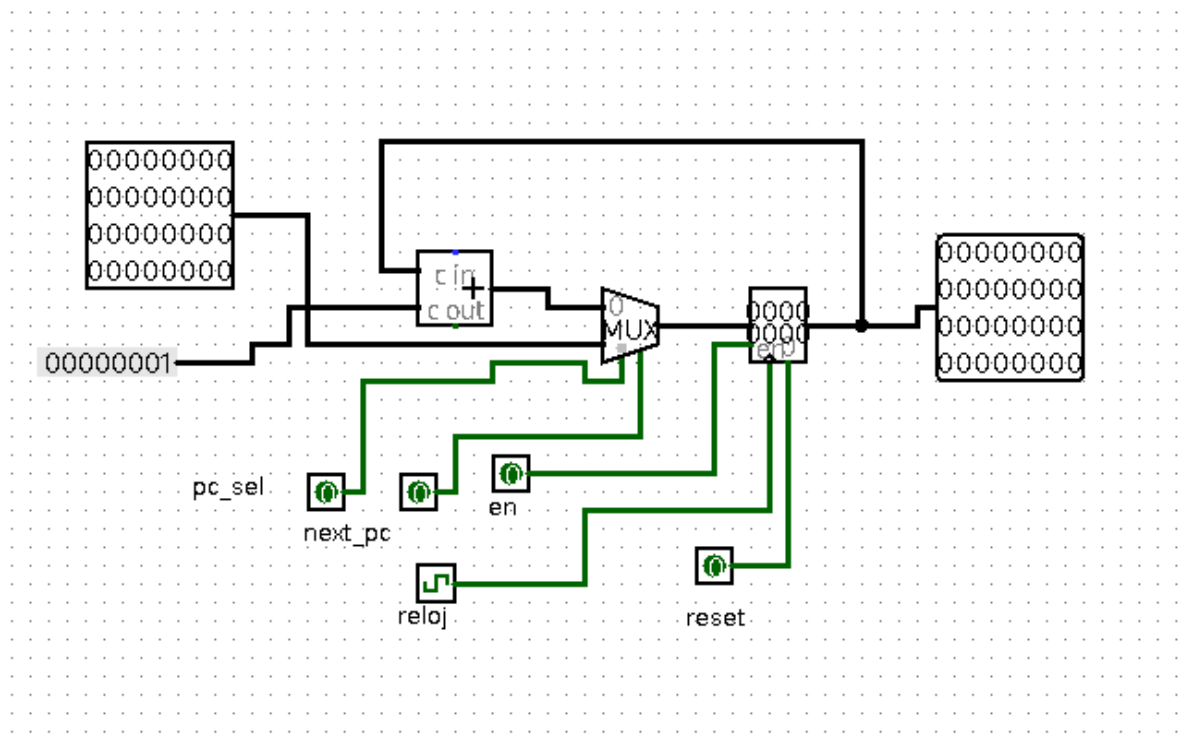
El PC se trata de un registro de 32 bits con el cual se guarda la dirección de la instrucción actual.

En donde se avanza automáticamente si no hay saltos, además carga una dirección externa cuando si hay instrucciones de salto, esto cuando se activa una señal.

Implementación:

Para su construcción se usó un registro de 32 bits con dos señales actuando de enable y clear, además se usó un sumador de 32 bits usando un multiplexor y una constante de igual forma y con su reloj para las pruebas.

Se tiene una señal para elegir entre PC+1 o una dirección externa, con un enable para detener el avance y un reset para reiniciar el contador.



En el código se tiene la variable global PC, que es la que almacene la dirección de la instrucción actual, mientras en el bucle principal se lee la instrucción en la posición PC y se incrementa, en las instrucciones de salto se asigna PC = inst.inmediato que es una carga manual de la dirección.

2. Memoria RAM y ROM con su selector

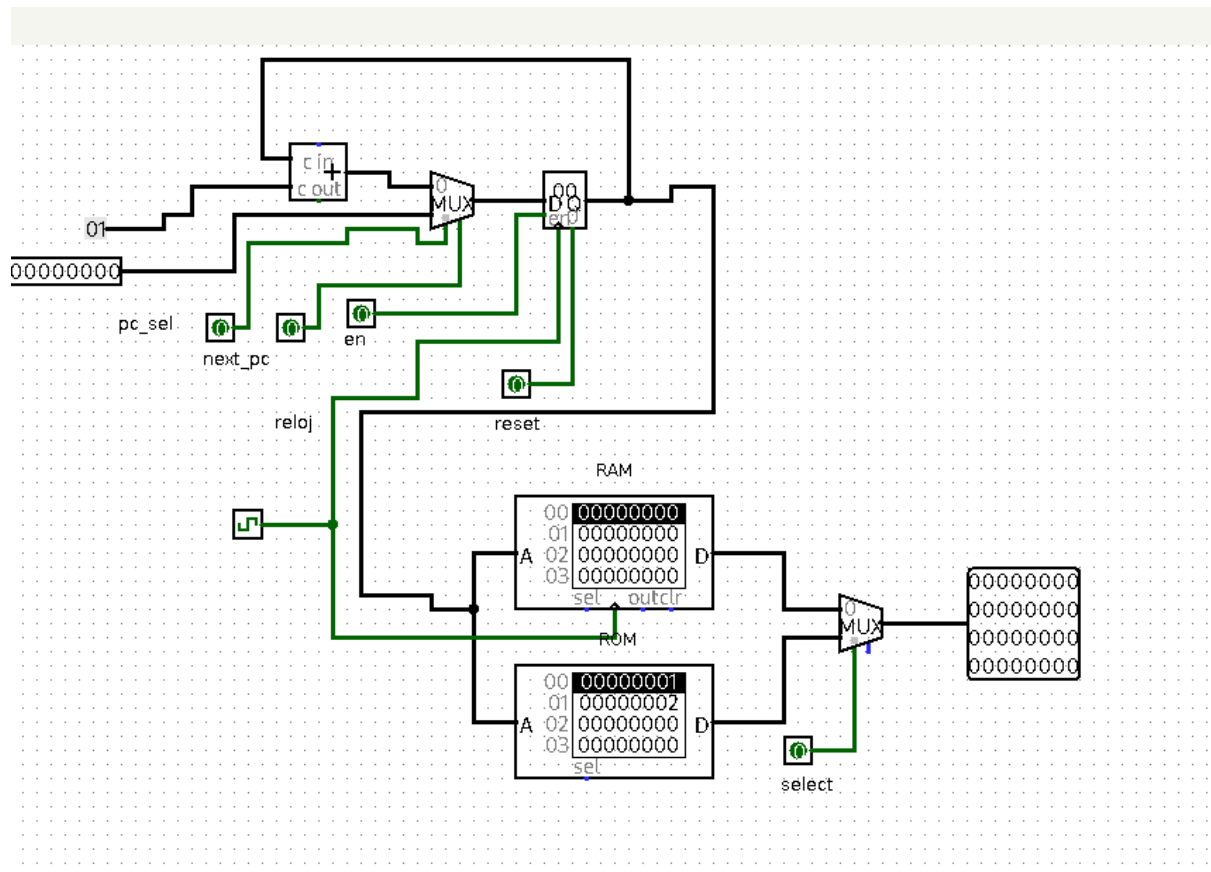
Se trata de un subsistema para la memoria que nos deja leer los datos tanto desde la RAM para escritura y lectura como de ROM solo para lectura. Usando un multiplexor de 2 a 1 con una señal para elegir cual de los dos datos se va a elegir para la salida

Implementación:

Para su realización se utilizaron dos módulos uno de RAM y el otro de ROM.

Para el de RAM es de 32 bits y se usan 8 bits para la dirección.

Mientras que el de ROM su contenido es fijo y se tiene que cargar manualmente.



RAM-ROM ya conectado al PC.

En el código se tienen como arrays `mem_Ins` que almacena las instrucciones ROM que se cargan al inicio y no se modifican después en la ejecución, además se tiene otro array memoria que almacena los datos del programa modificable con `lw` y `sw`. El multiplexor representa la separación de memoria y `mem_Ins`.

3. Unidad de Control

En la unidad de control se decodifica el código de cada operación siendo el opcode y además genera las señales de control que se usan en los demás componentes, se implementaron 8 instrucciones con las 4 dadas por la tabla siendo: `lw`, `sw`, `R-type`, `beq`.

Y se agregaron 4 adicionales, siendo: `addi`, `subi`, `andi` y `jump`.

Implementación:

Se utilizó un decodificador que activa una salida por instrucción, además las

Práctica 9: GALARISC-Microarquitectura
y Conjunto de Instrucciones

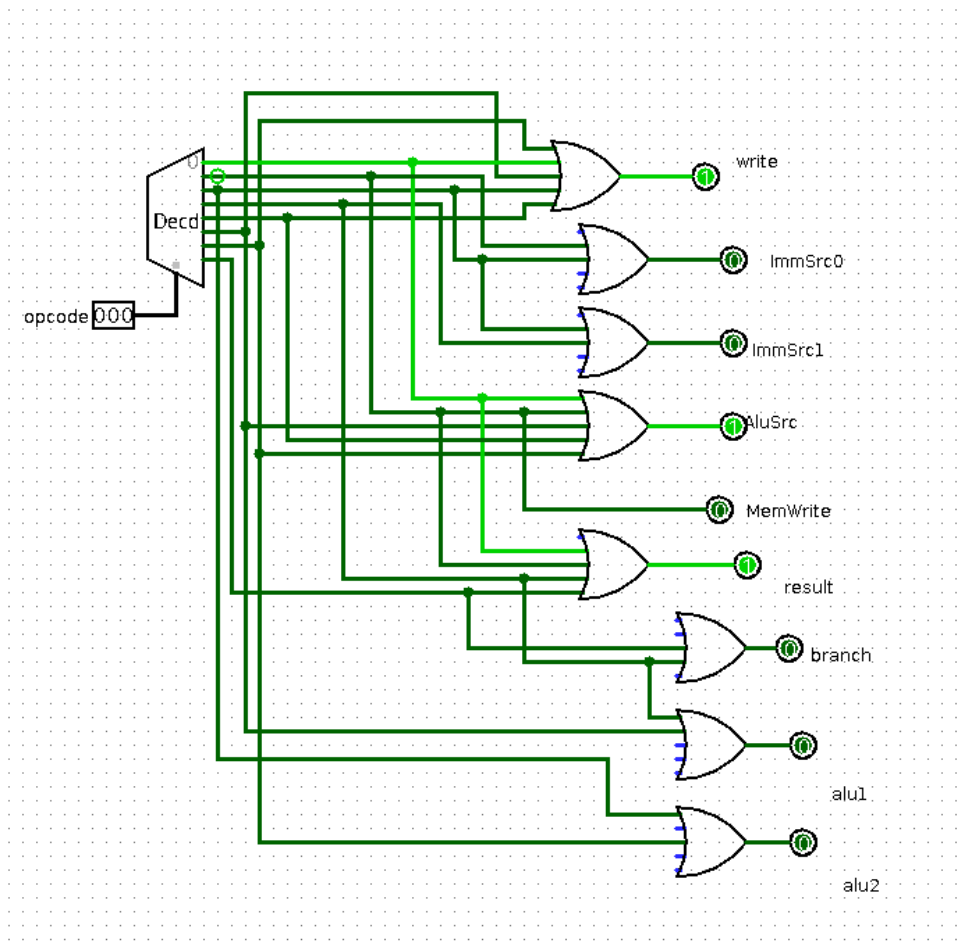
Angel Ismael Reyes Grimaldo
Sánchez Ortiz Diego

Organización y Arquitectura de Computadoras 2026-2

compuertas OR funcionan para combinar las salidas y producir cada señal según la tabla.

Tabla de las 8 instrucciones:

Instrucción	opcode	RegWrite	Imm Src	ALU Src	MemWrite	ResultSrc	Branch	ALUOp
lw	000	1	00	1	0	1	0	00
sw	001	0	01	1	1	1	0	00
R-type	010	1	11	0	0	0	0	10
beq	011	0	10	0	0	1	1	01
addi	100	1	00	1	0	0	0	00
subi	101	1	00	1	0	0	0	01
andi	110	1	00	1	0	0	0	10
j	111	0	xx	x	0	1	1	xx



En el código la cadena de if (strcmp(inst.name, "ADD") == 0) ... else if
(strcmp(inst.name, "AND") == 0) ...

Representa la codificación de la instrucción, con cada bloque if generando las acciones que corresponden, esas acciones son señales de control que Logisim toma como bits pero que C las toma como acciones.

La Unidad de Control recibe el opcode en el código es el string inst.name convertido a un número de 3 bits y produce las señales de control. El código C simula el efecto de esas señales.

4. ALU y Register File:

El ALU realiza las operaciones aritméticas y lógicas, entre ellas AND, OR, ADD, SUB, NEGATE, INCREMENT, DECREMENT y PASS. Además se generan las banderas cero, negativo y desborde.

Implementación:

Se partió del diseño de la práctica 4 y se amplió a 32 bits.

Se cambió la configuración de los sumadores multiplexores y compuertas lógicas para 32 bits.

Register File:

Se tiene un banco de 4 registros de 32 bits posiblemente escalables. Con dos puertos de lectura y un puerto para la escritura.

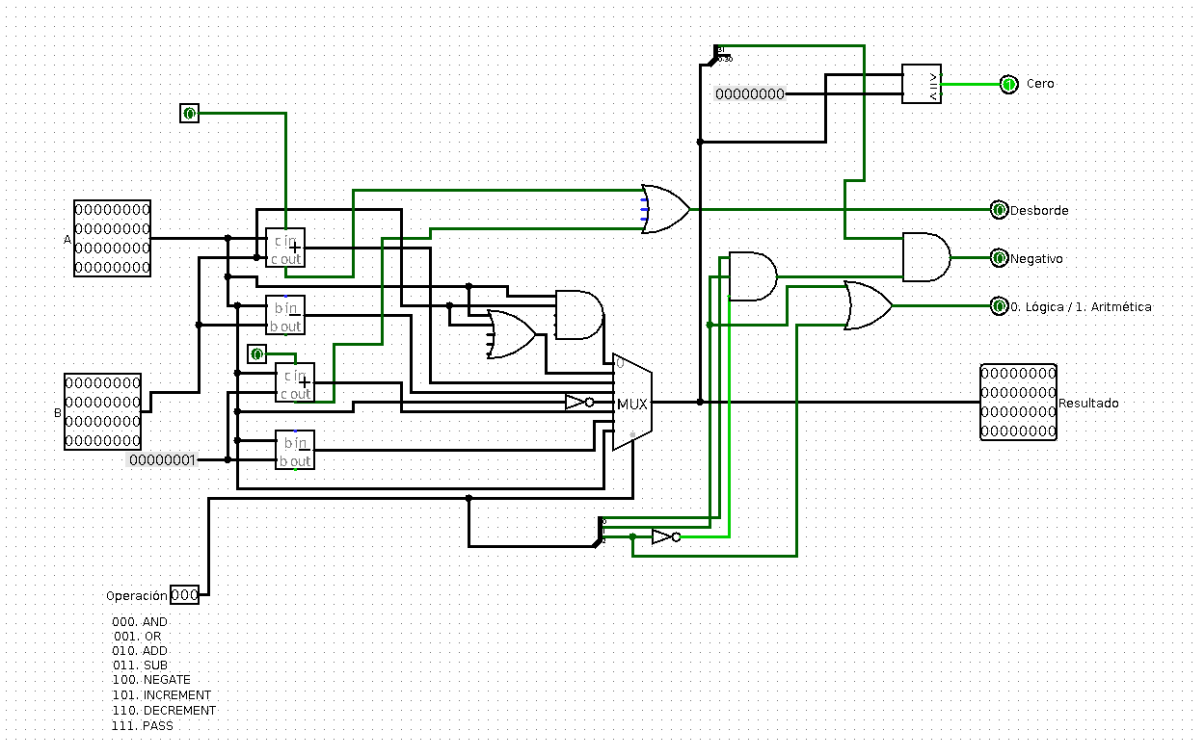
Implementación:

Para los registros se tiene un enable y un clock compartido, mientras que el demultiplexor se controla con una señal y sirve para activar el enable del registro que se seleccione.

Además tomamos dos multiplexores de 4 entradas para las lecturas seleccionados con dos señales.

Versiones punto flotante:

Se tienen copias de ALU y Register File de enteros, manteniendo el mismo ancho de banda. Pero en un procesador real los flotantes usaron circuitos diferentes pero nos limita Logisim a la hora de modificarlos.



Escalabilidad:

El ancho de banda que tenemos está limitado a 32 bits por Logisim, por ello ese es el tope que se usó, pero teóricamente es posible alcanzarlos usando splitters para dividir y combinar los buses. De forma similar el ALU de 64 bits se podría construir concatenando dos ALU's de 32 bits con propagación de su acarreo, este diseño es posiblemente escalable si la herramienta lo soportara.

Preguntas

1. A nivel Micro Arquitectura, ¿Cuál es la diferencia de diseño entre CISC y RISC?

CISC está caracterizado por tener un conjunto de instrucciones muy amplio que permite realizar operaciones sumamente complejas ya sea entre operandos o registros, pero a su vez eso hace que sea difícil su paralelización entre instrucciones. Mientras que RISC a pesar de tener un conjunto menor de operaciones estas se encuentran bien optimizadas lo que mejora la optimización en el rendimiento mediante pipelining.

Para el formato de instrucciones RISC trabaja con instrucciones de longitud fija (32 bits) los cuales están basadas en registros, por otra parte CISC usa un formato que permite desde 16 a 64 bits.

Para la unidad de control CISC usa unidades de control microprogramadas mientras que RISC solo usa control de cableado, así en CISC cada instrucción puede tomar muchos ciclos de reloj.

2. Tomando en cuenta el diseño de RISC-V, ¿Que partes del circuito se activan y qué hacen durante cada fase del ciclo FETCH-DECODE-EXECUTE WRITEBACK?

FETCH: El PC envía su valor a la memoria de instrucciones para obtener la instrucción y a su vez se incrementa el PC en 4, apuntando a la siguiente instrucción.

En esta parte los circuitos activos son: PC, Memoria de instrucciones y sumador de PC.

DECODE: En esta parte la instrucción que se obtuvo se envía al decodificador que da las señales de control y las líneas de datos, además se leen los valores de los registros en el Register File.

Los circuitos activos son: La unidad de control, el Register File, el extensor de signo y los multiplexores para la selección.

EXECUTE: Aquí dependiendo del tipo de instrucción, la ALU debe determinar el resultado, la dirección de memoria o la condición-dirección de destino para las instrucciones de salto.

Los circuitos activos son: ALU, multiplexores de operandos, la unidad de control de la ALU.

WRITEBACK:

Aquí el multiplexor es el que determina si se escribe de regreso al Register File ya sea el resultado de la ALU o el dato que se leyó desde memoria.

En instrucciones aritméticas-lógicas y de carga el resultado se escribe en el archivo de los registros. Para los altos se actualiza el PC con la dirección de destino.

Referencias:

- Block & Capital. (2025). *CISC vs RISC: Dos mundos en la arquitectura de procesadores*.
<https://blockandcapital.com/es/cisc-vs-risc-dos-mundos-en-la-arquitectura-de-procesadores/>
- IS603 Arquitectura de Computadoras. (2016). *Arquitectura CISC vs RISC*.
<https://is603arquicom2016.wordpress.com/arquitectura-cisc-vs-risc/>
- Tonmoy18. (2020, marzo 27). *Implementing our first instruction: Designing a RISC-V processor core*. GitHub Pages.
<https://tonmoy18.github.io/riscv-cpu-blog/2020/03/26/implementing-first-instruction.html>